Minor Project

Terraform AWS Infrastructure Setup

Step-by-step work document: provisioning a VPC, subnet, and EC2 instance with Terraform

1. Project Objective

Use Terraform (infrastructure as code) to provision a VPC, a public subnet, and an EC2 instance on AWS, then connect to the instance over SSH — the same end result as a manual console setup, but defined in code and created with a repeatable command instead of clicking through the console.

Tools Used

Terraform, AWS CLI, AWS EC2, AWS VPC.

Tasks

- Create a VPC
- Create a subnet
- Launch an EC2 instance using Terraform
- Connect to the instance through SSH

2. How Terraform Works (Quick Orientation)

Terraform reads `.tf` files describing the desired infrastructure, compares that to what currently exists (tracked in a local **state file**, `terraform.tfstate`), and makes the minimum changes needed to match. Three commands drive almost everything:

Command	What it does
terraform init	Downloads the AWS provider plugin. Run once per project folder, and again if providers change.
terraform plan	Shows what would be created/changed/destroyed, without doing it — a dry run.
terraform apply	Actually creates the infrastructure, after asking for confirmation.

3. Step 1: Install Terraform (Windows)

1. Download the **Windows AMD64** build from `developer.hashicorp.com/terraform/downloads`.
2. Create a permanent folder, e.g. `C:\terraform`, and extract the zip there so `terraform.exe` sits directly inside it.

3. Add `C:\terraform` to the Windows **Path** environment variable (Start menu → "Edit the system environment variables" → Environment Variables → Path → Edit → New → paste the folder path → OK on every window).
4. Open a **new** PowerShell window (existing windows won't see the PATH update) and confirm:

```
terraform -version
```

4. Step 2: Set Up AWS Credentials

Terraform authenticates to AWS the same way the AWS CLI does — through a saved credentials file, not a username/password.

1. Install the AWS CLI (Windows MSI installer) from `aws.amazon.com/cli` if not already installed. Confirm with `aws --version` in a new PowerShell window.
2. In the AWS Console: **IAM** → **Users** → **(your user)** → **Security credentials tab** → **Create access key**. Choose **Command Line Interface (CLI)** as the use case.
3. Copy both the **Access Key ID** and **Secret Access Key** (the secret is shown only once).
4. In PowerShell, run `aws configure` and enter the Access Key ID, Secret Access Key, a default region (e.g. `us-east-1`), and output format `json`.
5. Verify with:

```
aws sts get-caller-identity
```

A successful result prints your account ID, user ID, and ARN — confirming both the AWS CLI and Terraform can authenticate.

5. Step 3: Create a Key Pair for SSH

Terraform can reference an existing EC2 key pair, but it won't generate a safely-stored private key for you by default. The simplest approach is to create the key pair in the AWS Console first, the same way as in earlier projects, and have Terraform attach it by name.

1. **EC2** → **Key Pairs** → **Create key pair**.
2. Name: `terraform-key`. Type: RSA. Format: `.pem`.
3. Save the downloaded `terraform-key.pem` somewhere you'll remember — it's needed for the SSH step later.

6. Step 4: Write the Terraform Configuration

Create a new project folder, e.g. `C:\terraform-aws-project`, and inside it a single file named `main.tf` with the contents below.

main.tf

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

```
provider "aws" {
  region = "us-east-1"
}

# --- VPC ---
resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "terraform-vpc"
  }
}

# --- Subnet ---
resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block               = "10.0.1.0/24"
  map_public_ip_on_launch  = true
  availability_zone        = "us-east-1a"

  tags = {
    Name = "terraform-public-subnet"
  }
}

# --- Internet Gateway + routing, so the instance is reachable ---
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "terraform-igw"
  }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = {
    Name = "terraform-public-rt"
  }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}

# --- Security group: allow SSH only ---
resource "aws_security_group" "ssh" {
  name        = "terraform-ssh-sg"
  description = "Allow SSH access"
  vpc_id      = aws_vpc.main.id

  ingress {
    description = "SSH from my IP"
    from_port   = 22
    to_port     = 22
```

```
    protocol    = "tcp"
    cidr_blocks = ["YOUR_IP_ADDRESS/32"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "terraform-ssh-sg"
  }
}

# --- Latest Amazon Linux 2023 AMI, looked up automatically ---
data "aws_ami" "al2023" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }
}

# --- EC2 instance ---
resource "aws_instance" "web" {
  ami                    = data.aws_ami.al2023.id
  instance_type          = "t3.micro"
  subnet_id              = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.ssh.id]
  key_name               = "terraform-key"

  tags = {
    Name = "terraform-ec2-instance"
  }
}

# --- Output the public IP so it's easy to grab after apply ---
output "instance_public_ip" {
  value = aws_instance.web.public_ip
}
```

7. Step 5: Initialize, Plan, and Apply

From PowerShell, inside the project folder (`C:\terraform-aws-project`):

```
cd C:\terraform-aws-project
terraform init
```

This downloads the AWS provider plugin. You should see "Terraform has been successfully initialized!" at the end.

```
terraform plan
```

Review the plan output — it should list resources to add: the VPC, subnet, internet gateway, route table, route table association, security group, and EC2 instance (7 resources total), with a line reading "Plan: 7 to add, 0 to change, 0 to destroy."

```
terraform apply
```

Terraform shows the same plan again and asks for confirmation. Type **yes** and press Enter. This takes roughly 1–2 minutes. On success, it prints the `instance_public_ip` output value — copy that IP for the next step.

8. Step 6: Connect via SSH

From the same folder, or anywhere with the key file available:

```
ssh -i terraform-key.pem ec2-user@<instance_public_ip>
```

Type `yes` if asked about the host fingerprint on first connection. A successful connection lands on the instance's own prompt, e.g. `[ec2-user@ip-10-0-1-x ~]$`.

9. Deliverables: Screenshots

Capture the following for the report.

#	Screenshot	What to show
1	**terraform plan**	Shows the 7 resources to be created
	Screenshot removed from this public copy. The original capture contained account-specific AWS resource details, public IP information, or local machine identifiers. Add a new screenshot after masking identifying details.	
2	**terraform apply (+ "yes" confirmation)**	Shows resources created and the instance_public_ip output

#	Screenshot	What to show
	Screenshot removed from this public copy. The original capture contained account-specific AWS resource details, public IP information, or local machine identifiers. Add a new screenshot after masking identifying details.	
3	**AWS Console: VPC, subnet, and EC2 instance**	Confirms the resources exist, matching what Terraform created

Screenshot removed from this public copy.

The original capture contained account-specific AWS resource details, public IP information, or local machine identifiers.

Add a new screenshot after masking identifying details.

Screenshot removed from this public copy.

The original capture contained account-specific AWS resource details, public IP information, or local machine identifiers.

Add a new screenshot after masking identifying details.

#	Screenshot	What to show
	Screenshot removed from this public copy. The original capture contained account-specific AWS resource details, public IP information, or local machine identifiers. Add a new screenshot after masking identifying details.	
	Screenshot redacted for public sharing.	
4	**SSH connection**	Terminal showing a successful connection to the instance
	Screenshot removed from this public copy. The original capture contained account-specific AWS resource details, public IP information, or local machine identifiers. Add a new screenshot after masking identifying details.	

10. Troubleshooting

Problem	Fix
Downloaded binary runs oddly or triggers unexpected dialogs	Confirm the download matches your actual OS — a macOS build run on Windows (or the reverse) fails in confusing, indirect ways rather than a clear error.

Problem	Fix
`terraform` or `aws` not recognized after install	Close and reopen PowerShell — PATH changes don't apply to already-open windows.
`terraform apply` fails with a credentials/authentication error	Re-run `aws configure` and verify with `aws sts get-caller-identity`.
`terraform apply` fails: InvalidKeyPair.NotFound	The key_name in main.tf must exactly match a key pair that already exists in the same AWS region — create it in the EC2 console first (Step 3).
"... is not a valid CIDR block" on terraform plan	The security group's cidr_blocks needs a /32 suffix after the IP (e.g. 122.183.38.203/32), not just the bare IP address.
InvalidParameterCombination: instance type not eligible for Free Tier	Some accounts/regions only offer free-tier eligibility on t3.micro, not t2.micro. Change instance_type in main.tf to "t3.micro" and re-run terraform apply — Terraform will add only the instance, since the VPC, subnet, and security group from the first apply are unaffected.
SSH connection times out	Check the security group's ingress CIDR matches your current public IP; update main.tf and re-apply if it changed.
SSH: "Bad permissions" / "UNPROTECTED PRIVATE KEY FILE" on Windows	Windows file permissions on the downloaded .pem are too open for SSH's liking. Fix with: icacls "path\to\key.pem" /inheritance:r, then icacls "path\to\key.pem" /grant:r "$($env:USERNAME):(R)", then retry the ssh command.
Want to start over cleanly	Run terraform destroy to remove everything Terraform created, then fix the config and terraform apply again.

11. Clean Up

Unlike the console, clean-up here is one command — Terraform knows exactly what it created and removes all of it in the right order:

```
terraform destroy
```

Review the list of resources to be destroyed, type **yes** to confirm. This terminates the EC2 instance and removes the VPC, subnet, security group, route table, and internet gateway, so nothing is left running or charging.

12. Learning Outcomes

- Writing infrastructure as code with HashiCorp Configuration Language (HCL)
- Understanding the Terraform workflow: init, plan, apply, destroy
- Provisioning a VPC, subnet, internet gateway, and route table entirely from code
- Using a data source to look up the latest AMI automatically instead of hardcoding an ID
- Referencing an existing EC2 key pair from Terraform for SSH access
- Understanding Terraform's state file as its record of ownership of real infrastructure
- Comparing a code-defined, repeatable setup to the same setup built manually through the console